Counting function definitions with `scripts/count_functions.py`

Counts **named function definitions** in a crate's source tree. Rust and Python are counted accurately; other languages fall back to an approximate regex mode.

Run it

```
# one crate's source (the workspace default language is Rust)
scripts/count_functions.py crates/ragent-config/src

# the headline number only, tests excluded
scripts/count_functions.py crates/ragent-config/src --tests exclude

# machine-readable, for a report
scripts/count_functions.py crates/ragent-tui/src --json

# per-file counts and every counted name
scripts/count_functions.py crates/ragent-tui/src --list
scripts/count_functions.py crates/ragent-config/src/config.rs --names
```

A bare `path` may be a directory or a single file. The script is executable and runs on the system Python 3 (no third-party packages).

Options

Option	Meaning
`path`	Directory or file to scan (default `.`)
`--lang {auto,rust,python,generic}`	Language mode. `auto` (default) takes the language from the file extension for a file argument, and treats a directory as Rust
`--ext EXT`	Restrict to one extension; repeatable (`--ext .rs --ext .py`)
`--tests {include,exclude,only}`	Test functions in the headline total: `include` (default), `exclude`, or `only`
`--include-declarations`	Also count body-less `fn` declarations (`fn foo(&self) -> bool;`, `extern "C" { fn f(); }`)
`--include-macros`	Also count `fn` items written inside a `macro_rules!` body
`--closures`	Also report an approximate closure count (Rust only; never in the total)
`--list`	Per-file counts
`--names`	Every counted function, with line, category and name
`--json`	The whole report as JSON
`--no-default-excludes`	Do not skip `target/`, `node_modules/`, `.git/`, `__pycache__/`, `.venv/`
`--exclude-dir NAME`	Skip another directory name; repeatable

Option	Meaning
`--self-test`	Run the embedded fixture checks and exit without scanning anything

Exit status is `0` on success, `1` when `--self-test` finds a regression, and `2` on a usage or I/O error (for example a path that does not exist).

What counts as a function (Rust)

A record is created for every `fn` keyword followed by an identifier, which covers plain functions, inherent and trait methods, associated functions and nested functions:

```rust
fn plain() {}
pub fn public() {}
pub(crate) fn restricted() {}
pub async fn fetched() {}
pub async unsafe extern "C" fn ffi() {}
const fn limit() -> usize { 8 }
pub fn generic<T: Clone>(v: T) -> T { v }
pub fn where_clause<T>(v: T) -> T where T: Copy { v }
pub fn const_generic<const N: usize>() -> usize { N }
impl Holder { fn method(&self) {} }
trait Api { fn provided(&self) {} }
```

Each definition is counted once. The count is per `fn` item, not per line, so a multi-line signature, a `where` clause, a multi-line generic list, a `#[cfg]`-gated duplicate in a sibling module, or the same name implemented for two types all count once each.

Categories

Every hit is placed in exactly one category, and the headline total is the number of hits whose category is currently selected:

Category	How it is recognised	Counted by default
`production`	A `fn` item with a body and no test gate	yes
`test-attribute`	`#[test]`, `#[tokio::test]`, `#[rstest]`, `#[quickcheck]`, `#[test_case]`, `#[proptest]`	yes
`test-module`	Inside `#[cfg(test)] mod ...`, under a `#[cfg(test)]` item, or in a file gated by `#![cfg(test)]` (also `cfg(all(test, ...))`)	yes
`declaration`	No body: `fn f();` in a trait or `extern` block	no (`--include-declarations`)

Category	How it is recognised	Counted by default
`macro-body`	Inside a `macro_rules!` body, including the `$ty` fragments that generate one `fn` per type	no (`--include-macros`)

`--tests exclude` drops the two test categories; `--tests only` keeps just them. `--include-macros` is what makes a `macro_rules!` that stamps out one method per type comparable with a hand-written equivalent.

What is deliberately not counted

- `fn` in a line comment, a block comment (including nested ones), a doc comment, a doc-test example inside `///`, a string literal, a raw string, or a char literal — comments and literals are blanked out before matching, keeping line numbers intact.
- Closures (`|x| x + 1`, `move || ...`). Pass `--closures` for an approximate closure count; it is reported separately and never added to the total.
- Function-pointer *types* such as `fn(i32) -> i32` in a parameter, field or type alias: `fn` must be followed by an identifier to be a definition.
- `impl Trait for Type` blocks, struct fields and macro *invocations* (`my_macro!()`), which define nothing by themselves.

Assumptions and limits

- **Rust accuracy is textual, not a compiler.** The masking pass handles comments, nested block comments, normal/raw/byte strings, escapes and lifetimes; it is verified against a fixture that mixes all of them (see below). It is not a parser: it will not resolve conditional compilation, so a function present in two `cfg` variants of the same module is counted twice (once per definition).
- **`macro_rules!` bodies are excluded by default** because one body can stand for many generated functions. Use `--include-macros` when you want the *written* definitions, and remember the count is then "items written", not "functions compiled".
- **Generated code in `include!`d or build-script output is not seen**, since only the files under the given path are read.
- **Python** uses the stdlib `ast`, so nesting, methods, decorators and `async def` are exact. Functions whose name starts with `test` are put in the `test-attribute` category.
- **`generic` mode** (any other extension) matches a `def`/`fn`/`func`/ `function`/`fun`/`sub`/`proc` keyword at the start of a line. It does **not** strip comments or strings, so a definition shown inside a comment block can be counted. Treat those numbers as indicative only. Select extensions with `--ext` when a directory holds several languages.

Verification

The counting rules are pinned by an **embedded fixture** inside the script, so you can confirm the behaviour without a fixture file in the tree:

```
scripts/count_functions.py --self-test
# count_functions.py self-test: OK (rust + python fixtures)
```

It builds a Rust sample in memory that exercises every case above — line, block and nested block comments, doc examples, normal and raw strings, char literals and lifetimes, generics, `where` clauses, `const`

generics, `async`/`const`/`unsafe`/`extern "C"` functions, methods, trait and `extern` declarations, closures, `#[test]`, `#[tokio::test]`, `#[cfg(test)] mod` with a nested module, `cfg(all(test, ...))`, and two `macro_rules!` bodies — and asserts the exact per-category counts plus the absence of any comment/string false positive. It also checks the Python counter. A regression in the masker fails the self-test (exit status `1`) with the offending names listed, so run it after any edit to the script.

The directory walk is a thin layer over the same counters: point it at a throwaway sample under `target/temp/` to confirm the walk, the `--tests` modes and the extension filter end to end without touching the repository.

Cross-check against grep

For a Rust tree, the script's production count should equal a plain grep over the same files minus anything grep mistakes for a definition. On `crates/ragent-config/src` the script reports 349 production functions and a naive `rg '^\s*(pub\s+)?(async\s+|const\s+|unsafe\s+)*fn\s+[A-Za-z_]'` reports 350; the difference is the `fn fmt` inside `macro_rules! redacted_debug_impl`, which the script excludes by category. That equivalence — and the one-line explanation of any gap — is the right sanity check before quoting a headline number.